

Online Judge Platform

Prepared By: Kuswanth Tumma



Online Judge Platform — Architectural & Design Blueprint

[![License: MIT](https://img.shields.io/badge/License-MIT-yellow.svg)](https://opensource.org/licenses/MIT)

[![Node.js](https://img.shields.io/badge/Node.js-v18.x+-green.svg)](https://nodejs.org/)

[![React](https://img.shields.io/badge/React-v18.x-blue.svg)](https://reactjs.org/)

[![MongoDB](https://img.shields.io/badge/MongoDB-v6.0+-green.svg)](https://www.mongodb.com/)

[![Redis](https://img.shields.io/badge/Redis-v7.0+-red.svg)](https://redis.io/)

[![Docker](https://img.shields.io/badge/Docker-v24.0+-blue.svg)](https://www.docker.com/)

A scalable **Online Judge Platform** designed with production-oriented architectural principles. The system provides automated code execution, secure container sandboxing, hidden test case evaluation, and real-time user rankings.

Prepared By: Kuswanth Tumma



Table of Contents

[System Architecture Summary](#-system-architecture-summary)

[Key Architectural Capabilities](#-key-architectural-capabilities)

[Technology Stack](#-technology-stack)

[Repository Folder Structure](#-repository-folder-structure)

[Documentation Suite](#-documentation-suite)

[Architectural Design Q&A / Review Defense Guide](#-architectural-design-qa--review-defense-guide)

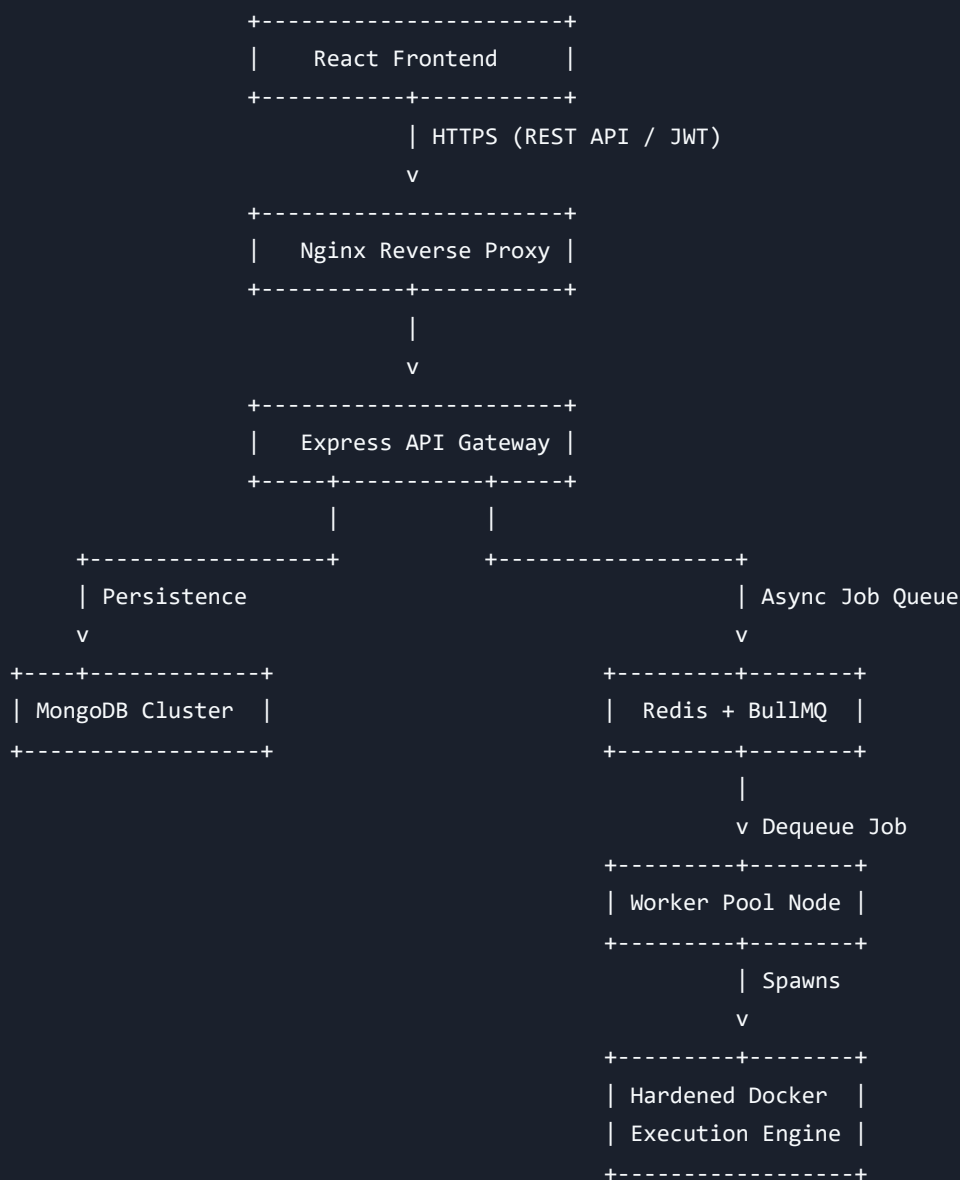
[Getting Started & Local Setup](#-getting-started--local-setup)

[License](#-license)



System Architecture Summary

The architecture decouples HTTP API request handling from compute-heavy code compilation and evaluation using an asynchronous queue pattern powered by **Redis and BullMQ**. Untrusted user code is designed to execute inside ephemeral, non-root **Docker containers** hardened with **Linux cgroups v2** (CPU/Memory limits), **seccomp** profiles (system call filtering), read-only root filesystems, and disabled network interfaces.



🌟 Key Architectural Capabilities

The architecture is designed to support:

🔒 **Secure JWT & RBAC Authentication:** Role-based access control separating regular coders from platform administrators.

⚡ **Asynchronous Code Execution:** Non-blocking job queues that handle bursty submission traffic without freezing API gateway instances.

🛡️ **Hardened Docker Sandboxing:** Ephemeral container environments isolated using custom `seccomp` system call filters, CPU/Memory cgroup limits, read-only root filesystems, and disabled network access.

📊 **Real-time Leaderboards & Submission History:** Efficient query execution for user submission audit logs, accuracy statistics, and global rankings.

🧪 **Hidden Test Case Evaluation:** Multi-testcase evaluation delivering standard verdicts (`Accepted` , `Wrong Answer` , `Time Limit Exceeded` , `Memory Limit Exceeded` , `Runtime Error` , `Compilation Error`).

Technology Stack

| Component | Technology | Rationale |

| :--- | :--- | :--- |

| **Frontend** | React 18, Vite, Tailwind CSS | Modern SPA architecture with fast Vite HMR bundling and Monaco Editor integration. |

| **Backend API** | Node.js, Express.js (TypeScript) | Non-blocking event loop provides high throughput for I/O bound routing with minimal overhead. |

| **Job Queue** | Redis, BullMQ | In-memory speed. BullMQ handles retries, concurrency limits, and job state events natively. |

| **Database** | MongoDB, Mongoose | Schema flexibility for dynamic testcases and high write throughput for submission logs. |

| **Execution Engine** | Docker, Linux cgroups, seccomp | OS-level container isolation for untrusted code execution. |

| **Reverse Proxy** | Nginx | SSL termination, rate limiting, and HTTP request routing. |

Repository Folder Structure

```

online-judge/
├── frontend/                                # React SPA Client
│   ├── src/
│   │   ├── components/                    # UI Components (Editor, Tables, Cards)
│   │   ├── pages/                        # Views (Home, ProblemDetail, Leaderboard)
│   │   ├── services/                     # API Client (Axios)
│   │   └── App.jsx                         # Root Route Setup
│   └── package.json
├── backend/                                # Express REST API Server
│   ├── src/
│   │   ├── controllers/                  # Route Controllers (Auth, Problem, Submission)
│   │   ├── middleware/                  # JWT Auth, RBAC Guard, Rate Limiter
│   │   ├── models/                      # Mongoose Schemas (User, Problem, Submission)
│   │   ├── routes/                      # Express Route Definitions
│   │   ├── services/                    # Business Logic Services
│   │   └── server.ts                     # API Gateway Entry Point
│   └── package.json
├── worker/                                # Asynchronous Queue Worker Node
│   ├── src/
│   │   ├── docker/                      # Docker Container Orchestrator
│   │   ├── evaluator/                   # Output Diffing & Verdict Engine
│   │   └── worker.ts                     # BullMQ Queue Processor Loop
│   └── package.json
├── docker/                                # Execution Sandbox Dockerfiles & Profiles
│   ├── cpp/Dockerfile                   # GCC Execution Environment
│   ├── python/Dockerfile                 # Python 3 execution sandbox
│   └── seccomp.json                      # Hardened Linux System Call Profile
├── docs/                                  # Architectural & Engineering Documentation
│   ├── 1_High_Level_Design.md            # Primary HLD Blueprint (Main Deliverable)
│   ├── 2_Low_Level_Design.md             # Low-Level MVC & Schemas
│   ├── 3_Software_Requirement_Specification.md # IEEE 830 Format SRS
│   ├── 4_API_Documentation.md           # OpenAPI 3.0 API Reference
│   ├── 5_Database_Design.md              # Database ERD & Indexing Strategy
│   └── 6_System_Diagrams.md              # 11+ Mermaid System Diagrams
├── docker-compose.yml                    # Local Development Deployment Setup
└── README.md                            # Repository Blueprint Overview

```

Documentation Suite

The complete engineering documentation package is located in the `docs/` directory:

1. 📄 **[High-Level Design (HLD) Blueprint](docs/1_High_Level_Design.md)** *(Main Submission Deliverable)*: A polished 10–15 page design blueprint covering system objectives, requirements, architecture, technology selection rationale, database schemas, REST APIs, Docker sandboxing, risk analysis, and future scope.
 2. 📄 **[Low-Level Design (LLD)](docs/2_Low_Level_Design.md)** *(Supporting)*: Class definitions, Mongoose schemas, controller interfaces, and worker execution logic.
 3. 📄 **[Software Requirement Specification (SRS)](docs/3_Software_Requirement_Specification.md)** *(Supporting)*: IEEE 830 specification listing all functional & non-functional requirements.
 4. 📄 **[API Documentation](docs/4_API_Documentation.md)** *(Supporting)*: OpenAPI 3.0 endpoints, authentication headers, request bodies, and error formats.
 5. 📄 **[Database Design](docs/5_Database_Design.md)** *(Supporting)*: ER diagrams, MongoDB collections, indexes, and sharding strategies.
 6. 📄 **[System Diagrams](docs/6_System_Diagrams.md)** *(Supporting)*: Mermaid diagrams covering Architecture, Sequence, Deployment, Class, Use Case, and Sandbox Isolation.
-



Architectural Design Q&A / Review Defense Guide

Use this section to confidently explain your design decisions during a review:

Q1: Why did you choose asynchronous execution (Redis + BullMQ) over synchronous execution?

Note

Answer: Code evaluation involves compilation and running against multiple test cases, which can take several seconds. If handled synchronously in an Express API request handler, API threads would block waiting for code execution, leading to HTTP timeouts and thread pool exhaustion during traffic spikes. Using Redis + BullMQ decouples request ingestion (instant 202 Accepted response) from processing, ensuring high API availability and smooth queue distribution.

Q2: How does the worker process communicate with Docker?

Note

Answer: The worker process runs on a Node.js runtime with access to the local Docker daemon socket (or Docker Engine CLI bindings). When a job is dequeued, the worker writes the code to a temporary directory `/tmp/sandbox_` and spawns a child process running `docker run` with volume mounts (`-v /tmp/sandbox_:/sandbox:ro`) and security flags (`--network none` , `--memory 256m` , `--security-opt profile=seccomp.json`).

Q3: Why did you choose Nginx as a reverse proxy?

Note

Answer: Nginx acts as a single entry point providing SSL/TLS termination, static file caching for the React SPA, IP-based rate limiting, and HTTP load balancing across API instances, keeping application servers focused purely on business logic.

Q4: How do cgroups and seccomp protect the host system?

Note

Answer: **cgroups (Control Groups)** limit physical hardware access (capping RAM to 256 MB and PIDs to 64 to prevent memory leaks and fork bombs). **seccomp (Secure Computing Mode)** filters kernel system calls, blocking untrusted code from invoking dangerous syscalls like `execve`, `socket`, or `ptrace`.

Getting Started & Local Setup

Quickstart with Docker Compose

1. Clone the repository:

```
`bash
git clone
cd online-judge
`
```

2. Configure Environment:

Create a `.env` file in the root directory:

```
`env
PORT=5000
MONGO_URI=mongodb://localhost:27017/online_judge
REDIS_HOST=localhost
REDIS_PORT=6379
JWT_SECRET=your_jwt_secret_key_here
`
```

3. Launch Local Environment:

```
`bash
docker-compose up --build -d
```



License

This project is licensed under the MIT License - see the LICENSE file for details.